

PROGRESS REPORT

Scientific Collaboration Network Analyzer

Reviewer and Institution Admin Workflow — Design, Implementation, and Verification

Prepared for: Project Mentor Review

Module: Full-Stack Feature Delivery — Role-Based Access Control and Reviewer Assignment

Milestone: 3 — Updated (follows Milestone 2 — Frontend/Backend Connectivity and Authentication)

Stack: FastAPI · SQLAlchemy · Alembic · PostgreSQL · Docker · Vanilla JS/HTML · pytest

Revision note: this document supersedes the original Milestone 3 report. It incorporates a further round of feature delivery, bug fixes, and an automated test suite completed after that report was first circulated, all within Milestone 3's original scope (Reviewer and Institution Admin functionality).

Table of Contents

1. Executive Summary
2. Position at the Start of This Milestone
3. Scope of This Milestone
4. Data Model Changes
5. Backend API Additions
6. Authorization and Institution-Scoping Logic
7. Frontend Integration
8. Environment Configuration Alignment
9. Automated Test Suite
10. Testing and Verification
11. Issues Encountered and Resolutions
12. Known Limitations and Suggested Next Steps
13. Conclusion

1. Executive Summary

Milestone 2 established a genuinely functioning connection between the ResearchSphere frontend and a live FastAPI and PostgreSQL backend, but it explicitly recorded two roles — Reviewer and Institution Admin — as present in the data model without any working functionality behind them. This phase of work closes that gap and, in the course of a further round of hands-on testing with the project mentor, closes a second set of gaps that only became visible once the first version of this feature was actually used: institution visibility on the Publications screen, an Institution Admin's ability to promote their own people into the Reviewer role, a scoped audit trail, and an automated regression test suite that already proved its worth by catching a real, pre-existing production bug unrelated to this milestone's original scope.

The work covered the full stack: new database tables and columns, new backend endpoints with role- and institution-aware authorization rules, the corresponding frontend screens, and — new in this update — a real pytest suite that exercises the backend through actual HTTP requests against a disposable database built from the project's real Alembic migrations. Every capability described in this report was exercised against a genuinely running PostgreSQL database rather than confirmed by code inspection alone, including six defects across the two rounds of work that inspection alone would not have caught.

Key outcomes:

- A new PublicationReview table records the full lifecycle of a reviewer assignment — who assigned it, who it was assigned to, any instructions left for the reviewer, the eventual decision, the reviewer's comments, and both timestamps.
- Institution Admin accounts are now linked to a real institution at the data-model level, and a subtler bug in how that institution was resolved for a promoted account was found and fixed.

- A System Admin can assign any reviewer to any publication; an Institution Admin can only assign reviewers who belong to their own institution, and only to publications authored within that institution — enforced server-side, not merely hidden in the interface.
- An Institution Admin can now promote an existing Researcher at their own institution directly to Reviewer (and back), without needing a System Admin, while being explicitly blocked from promoting anyone to Institution Admin or moving anyone between institutions.
- An Institution Admin now has a scoped view of the platform's audit log — the actions taken by their own institution's accounts — rather than no visibility at all.
- A reviewer's decision is not a cosmetic label: approving a submitted publication moves it to Published, and rejecting it returns it to Draft.
- A 26-test automated pytest suite now exercises the full workflow end-to-end against a real, migrated database, and was itself verified to catch real regressions — including a genuine, pre-existing production bug unrelated to this milestone that had been shipping undetected.

2. Position at the Start of This Milestone

The Milestone 2 report recorded, under its Known Limitations section, that Institution Admin accounts were not linked to a specific institution at the data-model level and that the role therefore defaulted to the first institution in the system regardless of who was actually logged in. The Reviewer role was present in the authentication system and could log in, but no endpoint, table, or interface existed for a reviewer to be assigned to anything, or to record a decision on anything. The frontend's “Review queue” screen from that phase displayed a fixed set of mock entries and its Approve and Reject actions produced only a confirmation toast, with no request ever sent to the backend.

This meant that, of the four roles the platform is designed to support, two — half of the total — had no working purpose beyond being able to authenticate. The objective set for this milestone was to bring both of those roles to full functional parity with the Researcher and System Admin roles already delivered, and — as the work progressed through direct use by the project mentor — to close the further gaps that surfaced once the first version of that functionality was actually exercised in practice rather than only through scripted tests.

3. Scope of This Milestone

The following capabilities were defined as in scope and were delivered:

- A reviewer-assignment workflow: a System Admin or Institution Admin can assign a specific Reviewer account to a specific publication, optionally with a note.
- A reviewer decision workflow: the assigned Reviewer can approve, request changes on, or reject the publication, with an optional comment, and the publication's own status responds accordingly.
- Institution scoping for Institution Admin accounts: a real institution association at the data-model level, and every review-related and staff-management endpoint restricted to that institution when the caller is an Institution Admin.
- Staff account management: the ability for a System Admin or Institution Admin to create new Reviewer accounts (and, for a System Admin only, new Institution Admin accounts), and to activate or suspend existing accounts within the caller's own scope.
- Role-switching within an institution: an Institution Admin can move an existing account of their own institution between Researcher and Reviewer, without System Admin involvement, subject to hard limits on what they can and cannot do.
- Visibility of institution ownership throughout the interface: which institution a publication belongs to, which institution a staff account is scoped to, and a scoped audit trail of an institution's own activity.

- The corresponding frontend screens: a real Review queue for Reviewers, a reviewer-assignment control on the Publications page, a reviewer-and-assignment management panel on the Institution page, and a scoped User Management page and Audit page for Institution Admins.
- An automated backend test suite covering authentication, the role-assignment authorization rules, the full reviewer workflow, and a smoke test across every public read endpoint.

Explicitly out of scope for this milestone, and carried forward as recorded in Section 12, were: automated browser-based end-to-end testing of the rendered interface, a fully verified docker compose flow from a completely clean machine, multi-reviewer consensus per publication, and full editing rights over an existing researcher's institution assignment by an Institution Admin.

4. Data Model Changes

4.1 A new `institution_id` column on the `User` table

Institution Admin and Reviewer accounts do not necessarily carry a full Researcher profile, so their institution could not be read from the same place a Researcher's institution is read from. A nullable `institution_id` foreign key was added directly to the `users` table, together with an `effective_institution_id` helper on the model.

This helper was revisited during the further round of testing described in this update: it originally checked a user's linked Researcher profile first regardless of their current role, which meant a Researcher promoted to Institution Admin or Reviewer and then explicitly assigned a different institution would still silently resolve to their old, stale Researcher-profile institution. It now checks role first — Researchers read their institution from their Researcher profile, every other role reads it from the direct assignment — which is the correct behaviour and was re-verified end-to-end, described further in Section 11.

4.2 A new `PublicationReview` table

A new table, `publication_reviews`, was introduced via an Alembic migration to record the full lifecycle of a reviewer assignment rather than reducing it to a single status flag on the publication itself. Each row records: the publication being reviewed, the reviewer it was assigned to, who assigned it, an optional instructional note, the decision status, the reviewer's decision comments, and both the assignment and decision timestamps.

4.3 A missing `progress_percentage` column on `Project` (unrelated to this milestone's scope, found and fixed during it)

Interactive testing during this update surfaced a pre-existing defect: the `Project` model has long declared a `progress_percentage` column, but no migration had ever actually created it in the database. On any database built by applying migrations one at a time — the normal path for an existing deployment, as opposed to being created fresh from the current models — `GET /projects` failed outright with a 500 error. A migration adding the missing column was written and applied, and is detailed further in Section 11.

5. Backend API Additions

A new router, mounted at `/reviews`, was added alongside the existing routers for researchers, publications, and institutions:

- `GET /reviews/reviewers` — returns the accounts eligible to be assigned as a reviewer; a System Admin sees every reviewer on the platform, an Institution Admin sees only reviewers belonging to their own institution.
- `POST /reviews` — assigns a reviewer to a publication, with a note. Rejects a duplicate pending assignment to the same reviewer, and — for an Institution Admin caller — rejects any attempt to assign across an institution boundary.
- `GET /reviews` — lists assignments, scoped to the caller's institution for an Institution Admin and unscoped for a System Admin, with optional filtering by decision status.
- `DELETE /reviews/{id}` — cancels an assignment that has not yet been decided; a decided assignment cannot be cancelled, preserving the record.

- GET /reviews/mine — a Reviewer's own queue, both pending and previously decided.
- POST /reviews/{id}/decision — records the Reviewer's decision and comments; can only be called by the reviewer the assignment actually belongs to, and only once per assignment.

The existing admin router was extended further in this update:

- POST /admin/staff — onboards Reviewer and Institution Admin accounts, mirroring the account-creation and password-reset-email pattern already established for researcher onboarding.
- PATCH /admin/users/{id} — an Institution Admin caller can now switch an existing account of their own institution between Researcher and Reviewer, in addition to the previously delivered activate/suspend capability. The same endpoint continues to reject any attempt by an Institution Admin to promote someone to Institution Admin or System Admin, or to reassign a user to a different institution.
- GET /admin/audit-logs — previously System Admin only; an Institution Admin caller now receives a scoped result set containing only actions taken by accounts belonging to their own institution, rather than being denied access outright.
- GET /researchers — now filters to accounts whose current role is still Researcher, so an account that has been promoted to Reviewer or Institution Admin no longer appears in the researcher-browsing list, while its existing authorship links on past publications and projects remain fully intact.

Suspending a Reviewer account that still has an undecided assignment now also cancels that assignment automatically, rather than leaving it silently pending against an account that can no longer act on it — the admin who suspended the account still needs to assign a replacement, but no longer needs to separately notice and cancel the stale assignment first.

6. Authorization and Institution-Scoping Logic

The central design decision of this milestone was that institution scoping had to be enforced by the backend on every relevant request, not only reflected in what the frontend chooses to display. An Institution Admin account making a direct API call — bypassing the interface entirely — is subject to exactly the same restrictions a well-behaved frontend would have presented to them. The rules applied consistently across the relevant endpoints are:

- An Institution Admin may only assign reviewers who belong to their own institution.
- An Institution Admin may only assign reviewers to publications whose corresponding author belongs to their own institution.
- An Institution Admin may only manage (activate, suspend, and — as of this update — switch between Researcher and Reviewer) accounts within their own institution.
- An Institution Admin may never change a user's role to Institution Admin or System Admin, and may never move a user to a different institution — both remain System-Admin-only actions, verified by deliberately attempting each and confirming rejection.
- An Institution Admin's audit-log view is restricted to actions taken by accounts belonging to their own institution.
- A System Admin is exempt from all institution boundaries and can act across the whole platform, consistent with its role in Milestones 1 and 2.

Every one of these rules was exercised directly by attempting the disallowed action and confirming it was rejected, not merely reasoned about, as described in Section 10.

7. Frontend Integration

The frontend changes followed the same principle established in Milestone 2: reshape live API responses into the structure the existing rendering functions already expect, rather than rebuilding the interface. Building on the screens delivered earlier in this milestone, the following areas were added or reworked in this update:

- The Publications screen now shows which institution each publication belongs to, resolved the same way the backend resolves it for authorization purposes. For an Institution Admin, the Assign control is disabled with an explanatory tooltip on any publication that is not theirs to assign, rather than allowing the click and only reporting the rejection afterward.
- The Edit User modal gained a visible, editable Institution field for staff roles (Institution Admin, Reviewer), so a System Admin can see and set which institution a promoted account belongs to — previously invisible in the interface even though the backend already tracked it.
- Institution Admin accounts now have their own scoped User Management page, listing only accounts at their own institution, with the ability to switch a person between Researcher and Reviewer and to activate or suspend them — with search-by-name/email/role and pagination added to this page for both roles.
- Institution Admin accounts now have scoped access to the Audit & Compliance page, showing only their own institution's activity, with the underlying raw table browser (a System Admin-only diagnostic tool) correctly withheld from them.
- The Institution page's reviewer panel now carries a short inline note pointing an Institution Admin toward the User Management page to promote an existing researcher, rather than only offering the “create a brand-new account” path, since both now exist and were otherwise easy to conflate.
- The Profile page now shows the signed-in account's institution for every role, not only Researchers — System Admin accounts are shown as platform-wide rather than blank.

8. Environment Configuration Alignment

Milestone 2 documented remapping the backend to port 8001 and the database container to port 5433 to avoid local port conflicts. During the earlier part of this milestone it was found that the checked-in defaults — the `.env.example` template, the application's internal configuration fallbacks, and the frontend's `API_BASE_URL` constant — still referenced the original ports 8000 and 5432. These were brought into alignment, and a working `.env` is now included directly rather than requiring a manual copy-and-edit step.

During the further round of hands-on setup covered by this update, two additional environment issues surfaced and were resolved: the frontend's `API_BASE_URL` was standardised on the explicit `127.0.0.1` address rather than `localhost`, avoiding a Windows-specific IPv6 resolution issue; and the project's setup guidance now explicitly calls for serving the frontend over a local HTTP server rather than opening it directly from disk, since Chrome on Windows blocks a `file://` page's requests to a local server as a private-network security measure. Both are detailed in Section 11.

9. Automated Test Suite

The original Milestone 3 report recorded, as a known limitation, that verification had been carried out through direct scripted API calls rather than an automated, repeatable test suite. That gap is closed in this update. A pytest suite of 26 tests was added under `backend/tests/`, covering:

- The 2-step login/OTP flow, including correct rejection of a wrong password and a wrong OTP code.
- Role-assignment authorization: a System Admin promoting an account to any role; an Institution Admin promoting their own researcher to Reviewer; an Institution Admin being blocked from promoting to Institution Admin, from touching another institution's accounts, and from reassigning a user's institution; and the account-listing endpoint correctly scoping results to an Institution Admin's own institution.
- The full reviewer workflow: assignment, the reviewer seeing it in their own queue, an approval publishing the article, a rejection returning it to draft, duplicate-assignment rejection, cross-institution assignment rejection on both the reviewer side and the publication side, a reviewer being blocked from deciding someone else's assignment, a decided review being immutable, and cancellation of a still-pending assignment.

- Researcher-list visibility: a promoted account disappearing from and reappearing in the researcher-browsing list, and confirmation that its authorship record on an existing publication survives the role change unaffected.
- A smoke test hitting every public read endpoint (institutions, researchers, publications, projects, conferences, health) with real seeded data rather than an empty database, specifically because an empty table cannot reveal a missing or misconfigured column — exactly the class of defect described in Section 4.3.

A deliberate design decision was made about how the test database's schema is built. The first version of this suite built its schema directly from the current SQLAlchemy models (`Base.metadata.create_all`), which is the common, faster approach. On review, this was identified as a meaningful weakness: it reads schema straight from the models and would therefore never be able to catch a defect where a model declares a column that no migration actually creates — precisely the `progress_percentage` defect described in Section 4.3 and Section 11. The suite was changed to instead run the project's real Alembic migration chain against a disposable test database for every test session.

This was not left as an untested assumption. The migration that adds the `progress_percentage` column was temporarily removed from the project, the suite was re-run, and the corresponding test failed with the exact same `UndefinedColumn` error a real user would have encountered — while every other test remained unaffected. The migration file was then restored and the full suite re-confirmed passing. This is offered as direct evidence that the suite would catch this specific class of regression in the future, rather than a claim taken on faith.

Building this also surfaced a second, related defect: the project's `alembic/env.py` unconditionally overrode any supplied database URL with the application's own configured connection string, which meant the test suite's first attempt to point migrations at a disposable test database was silently discarded and migrations ran against the wrong target. This was resolved with a `TEST_DATABASE_URL` environment-variable check that only the test suite sets, leaving normal developer use of Alembic against the real development database completely unaffected — confirmed by re-running a normal alembic current against the development database afterward.

The suite can be run with `pip install -r requirements.txt` followed by `pytest` from the backend folder, requires only that Postgres already be running, and creates and tears down its own disposable database automatically — real developer data is never touched. Every individual test also runs inside its own database transaction that is rolled back afterward, so tests cannot see or interfere with one another's data.

10. Testing and Verification

Consistent with the verification standard set in Milestone 2 — that a capability is not considered delivered until it has been independently exercised, not merely coded — every capability in this milestone, across both rounds of work, was tested against a genuinely running PostgreSQL database rather than confirmed by reading the code. The verification sequence was:

- A fresh PostgreSQL database was provisioned and every Alembic migration — including the two added since the original report — was applied to it, both upgrading and, separately, downgrading, to confirm each migration is cleanly reversible.
- The project's existing seed script and demo-account script were run against that database to populate realistic institutions, researchers, and publications, and to create memorable demo accounts for each of the four roles.
- The backend server was started against that database and every endpoint — both from the original scope and the additions in this update — was exercised with real HTTP requests: logging in as each role, reading the institution-scoped reviewer list, assigning a reviewer, attempting a duplicate assignment, attempting a cross-institution assignment, submitting a decision, attempting to decide the same assignment twice, promoting a researcher to reviewer and confirming their institution carried over correctly, attempting to escalate a promotion to Institution Admin, reading a scoped audit log, and confirming a promoted account disappears from and reappears in the researcher list correctly.
- The authorization boundaries described in Section 6 were tested from the losing side deliberately — for example, having an Institution Admin attempt to assign a reviewer belonging to a different institution, or attempt to manage an account outside their own institution — to confirm each was correctly rejected rather than merely assuming it would be.

- Referential integrity was confirmed directly: deleting a Reviewer account that had an active review assignment was confirmed to remove the associated review row cleanly, without a foreign-key error, consistent with the cascade behaviour defined on the table.
- The full sequence above was repeated multiple times end-to-end, against a completely fresh database with no manual intervention, to confirm the process is reproducible from a clean checkout and not dependent on any state left over from earlier debugging.
- The automated test suite described in Section 9 was run to a clean pass (26/26) as the final verification step for this update, and was itself validated by deliberately breaking two different pieces of underlying logic — a swapped approve/reject condition, and a removed migration — and confirming the suite failed in each case, before confirming it passed again once each was restored.

This process is what surfaced the defects recorded in Section 11 — none would have been caught by code review alone, since all were failures that only manifest when the code executes against an actual database engine or a real, unscripted user session.

11. Issues Encountered and Resolutions

As in the Milestone 2 report, practical issues encountered are recorded here together with their diagnosis and resolution, since working through them was a material part of the effort involved and is representative of the verification standard applied throughout. The first seven rows were recorded in the original Milestone 3 report; the remaining rows were found and resolved during the further round of work covered by this update.

Issue	Resolution
New enum type created with lowercase values	The new review-status database enum was initially created with lowercase values (pending, approved, ...). Live testing showed every other enum in the schema instead stores the Python model's uppercase member names. Left uncorrected, every write to a review's status would have failed in production. The migration was corrected to match the schema's existing convention, and re-verified against a clean database.
Enum type created twice in one migration	The same migration attempted to create its enum type once explicitly and a second time implicitly while creating the table, which PostgreSQL correctly rejected as a duplicate object within the same transaction. Resolved by disabling the implicit creation and keeping the single explicit, existence-checked creation.
Manual downgrade/upgrade cycle lost demo account data	During interactive debugging, manually downgrading and re-upgrading the migration dropped and recreated the new institution_id column, which discarded values set on it moments earlier for the demo accounts. This was specific to manual mid-session schema surgery, not the normal one-time migration path, and was resolved by re-running the demo account values; a full clean-database run afterwards confirmed a normal single upgrade never encounters this.
Backend unreachable at http://localhost:8001 despite the server running	The backend was confirmed healthy at 127.0.0.1:8001 but unreachable at localhost:8001 from the same machine — a known Windows behaviour where localhost can resolve to an IPv6 address the server is not listening on. Resolved by standardising the frontend's API_BASE_URL on the explicit 127.0.0.1 address, which is unambiguous on every platform.
Frontend still could not reach the backend after the above fix	Even after correcting the address, requests were being blocked by the browser rather than refused by the server. The frontend was being opened directly from disk (a file:// address); Chrome on Windows blocks such pages from making local-network requests as a security measure,

	independent of whether the server is reachable. Resolved by serving the frontend over a local HTTP server instead of opening the file directly.
Checked-in port defaults did not match the documented local setup	The .env.example template, the application's internal configuration fallbacks, and the frontend's API_BASE_URL constant still referenced the project's original ports (8000 and 5432) rather than the 8001 and 5433 ports documented for local development, so a fresh checkout would not match the setup instructions. All three were brought into alignment, and a working .env is now included directly rather than requiring a manual copy-and-edit step.
Missing institution_id column against a pre-existing database	An existing local database created before this milestone's migration was written did not yet have the new institution_id column, producing an UndefinedColumn error on login. This is expected behaviour for any schema change against a database that has not yet been migrated, and was resolved by running alembic upgrade head against that database before restarting the server.
A Researcher promoted to Institution Admin silently kept the wrong institution	effective_institution_id was resolving a user's institution by checking their linked Researcher profile first, regardless of current role. A Researcher promoted to Institution Admin or Reviewer and then explicitly assigned a different institution would still be scoped to their old, stale Researcher-profile institution, because that check ran first and never looked at the newly assigned value. Fixed by checking role first: Researchers read their institution from their Researcher profile, every other role reads it from the direct assignment. Verified by promoting a test account, assigning it a different institution, and confirming /auth/me and every scoping check reflected the new institution rather than the old one.
GET /projects returned HTTP 500 on a step-by-step migrated database	Discovered during interactive testing, not part of this milestone's original scope: the Project model declares a progress_percentage column, but no Alembic migration had ever actually created it. Any database built by running migrations one at a time — rather than created fresh from the current models — was missing the column, and every request to GET /projects failed with an UndefinedColumn error. A migration adding the missing column was written, applied, and the endpoint was re-verified to return real project data afterward.
Docker container name conflicts during re-setup	Restarting the local Docker environment after a port remapping hit two separate "container name already in use" conflicts (scan_pgadmin, then a stale scan_postgres left in a Created-but-never-started state from the first conflict). Both were resolved by explicitly removing the conflicting container before re-running docker compose up, which is now called out directly in the setup guidance given for this project.
Test database URL override was silently ignored	While building the automated test suite (Section 9), the first attempt pointed Alembic at a disposable test database by passing an explicit URL to the migration command, but alembic/env.py unconditionally overwrote that with the application's own configured database URL, so migrations were silently running against the wrong database and every test failed. Resolved by adding a TEST_DATABASE_URL environment-variable check that only the test suite sets, leaving normal developer use of Alembic completely unaffected.

12. Known Limitations and Suggested Next Steps

The following gaps remain and are consciously deferred, in the same spirit as the limitations recorded at the end of Milestone 2. Several items recorded as limitations in the original Milestone 3 report have since been resolved and are not repeated here; where a related but narrower gap remains, it is noted:

- Verification of the reviewer and institution-admin workflow, and of the automated test suite itself, was carried out through direct API calls and scripted HTTP requests rather than an automated browser-based end-to-end test suite that drives the rendered interface. The pytest suite added in this update is a genuine regression safety net for the backend, but it does not click through the actual HTML/JS interface a user sees.
- The docker compose flow specifically was not watched succeeding start-to-finish on a completely clean machine within the environment used to carry out this milestone's verification, since no Docker daemon was available there; a native PostgreSQL instance on the same ports was used as a substitute for verification purposes. The underlying SQL, migrations, and API behaviour are identical either way, but this is recorded honestly as a gap in how it was verified, not only in what was built.
- Suspending a Reviewer account with a pending assignment now automatically cancels that assignment rather than leaving it silently stuck, but no automatic reassignment to a replacement reviewer occurs; an admin still has to assign a new reviewer manually.
- An Institution Admin can promote an existing researcher to Reviewer, and can onboard new researchers into their own institution, but still cannot reassign an existing researcher's institution or edit a researcher's profile beyond creation; both remain System-Admin-only actions.
- The reviewer-decision workflow still supports a single reviewer per assignment; multi-reviewer consensus or a required second opinion is not modelled.
- Two parallel paths now exist for an Institution Admin to get a new Reviewer into the system — inviting a brand-new account, or promoting an existing researcher. Both work correctly and a clarifying note now links them together in the interface, but they have not been consolidated into a single flow.

These are recorded as a prioritised backlog for a subsequent phase of work rather than as defects in what has been delivered.

13. Conclusion

This milestone brought the Reviewer and Institution Admin roles from authentication-only placeholders to fully functional parts of the platform, closing the specific gap recorded at the end of Milestone 2, and then closed a further set of gaps that only became apparent once that functionality was put in front of the project mentor and used directly rather than only tested by script. An Institution Admin is now genuinely scoped to their own institution at the data-model level, can manage their own people — including promoting a researcher to reviewer — without System Admin involvement, and can see a scoped view of their own institution's activity. A Reviewer's decision has a real, verified effect on the publication it concerns.

A meaningful addition in this update is the automated test suite: rather than relying solely on manual, scripted verification repeated by hand each time, the project now has a 26-test regression suite that runs the real migration chain and was proven, not merely claimed, to catch the specific class of defect — a model declaring a column no migration ever created — that had already shipped undetected once in this project's history.

As with Milestone 2, the standard applied throughout was that a capability is not considered complete until it has been demonstrated working against a real, running system: a genuinely provisioned PostgreSQL database, a live backend process, and real HTTP requests exercising both the successful path and the paths that are supposed to fail. Across both rounds of this milestone, that process surfaced eleven distinct issues that would not have been visible from source code alone, all of which were corrected and re-verified before this update was considered complete.